Republic of the Philippines
College of Engineering and Information Technology
DEPARTMENT OF COMPUTING AND LIBRARY INFORMATION SCIENCE

CS ELEC 01 – COMPUTATIONAL SCIENCE
Final Activity 1: Case Studies on Numerical Methods

GROUP 13
Members:
Charles Cabrera
Tayba B. Abdullah
Erika G. Uyasan

CASE STUDY 4: Water Tank Filling Rate Analysis
Made by/Reporter: Charles Cabrera

Scenario:
A water tank is being filled, and sensors record the volume of water (in liters) over time (in minutes).
Engineers want to determine:

1. The rate at which water is entering the tank
2. The total volume accumulated over time

Computational Task
Students must:

1. Estimate the rate of inflow (derivative).
2. Compute total volume change (integration).
3. Analyze where the flow is constant or variable.

1. Key Idea
Volume V(t) = how much water is in the tank
Derivative V'(t) = how fast water is entering (flow rate)
Integration = total accumulated water

In short:
Volume = “how much water is there”
Flow rate = “how fast it’s filling”

2. FLOW RATE (Derivative using central difference)

Formula used:

Formula used: $V'(t) \approx \frac{V(t+h)-V(t-h)}{2h}$

Given: h = 2

Compute:

At t = 2
= (V(4) - V(0)) / 4

= (110 - 0) / 4 = **27.5 L/min**

At t = 4
= (210 - 40) / 4 = **42.5 L/min**

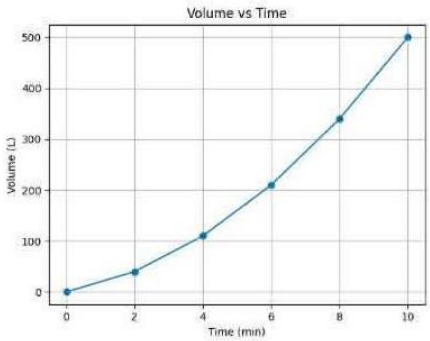
At t = 6
= (340 - 110) / 4 = **57.5 L/min**

At t = 8
= (500 - 210) / 4 = **72.5 L/min**

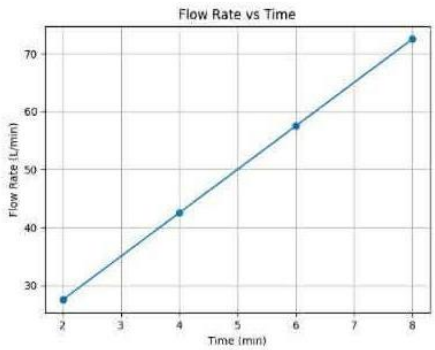
Final Flow Rate Table

Time = **2**
Flow Rate (L/min) = **27.5**
Time = **4**
Flow Rate (L/min) = **42.5**
Time = **6**
Flow Rate (L/min) = **57.5**
Time = **8**
Flow Rate (L/min) = **72.5**

Volume vs Time Graph



Flow Rate vs Time Graph



3. TOTAL VOLUME (Integration – Trapezoidal Rule)

Formula:

Formula: $\int_0^{10} V(t)\,dt \approx \sum \frac{V_i+V_{i+1}}{2} \cdot \Delta t$

Compute each interval (Δt = 2):

(0+40)/2 ×2 = 40
(40+110)/2 ×2 = 150
(110+210)/2 ×2 = 320
(210+340)/2 ×2 = 550
(340+500)/2 ×2 = 840

Total:
= **1900 liters**

4. ANALYSIS

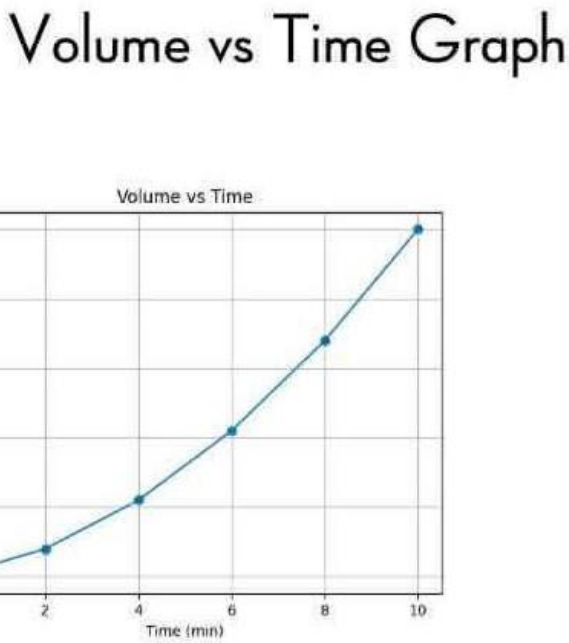
❌ Flow is NOT constant
✅ Flow rate is Increasing over time
System is accelerating (nonlinear)
Fastest inflow: at t = 8 min

Source Code (Volume vs Time):

```
Import matplotlib.pyplot as plt


# GIVEN DATA

Time = [0, 2, 4, 6, 8, 10]
Volume = [0, 40, 110, 210, 340, 500]


# COMPUTE FLOW RATE (Central Difference)

Flow_time = []
Flow_rate = []

H = 2

For I in range(1, len(time)-1):
    Rate = (volume[i+1] – volume[i-1]) / (2*h)
    Flow_time.append(time[i])
    Flow_rate.append(rate)

# GRAPH 1: Volume vs Time
Plt.figure()
Plt.plot(time, volume, marker='o')
Plt.title("Volume vs Time")
Plt.xlabel("Time (minutes)")
Plt.ylabel("Volume (Liters)")
Plt.grid()


# GRAPH 2: Flow Rate vs Time
```

```
Plt.figure()
Plt.plot(flow_time, flow_rate, marker='o')
Plt.title("Flow Rate vs Time")
Plt.xlabel("Time (minutes)")
Plt.ylabel("Flow Rate (L/min)")
Plt.grid()


# SHOW GRAPHS
Plt.show()
```

Output (image):

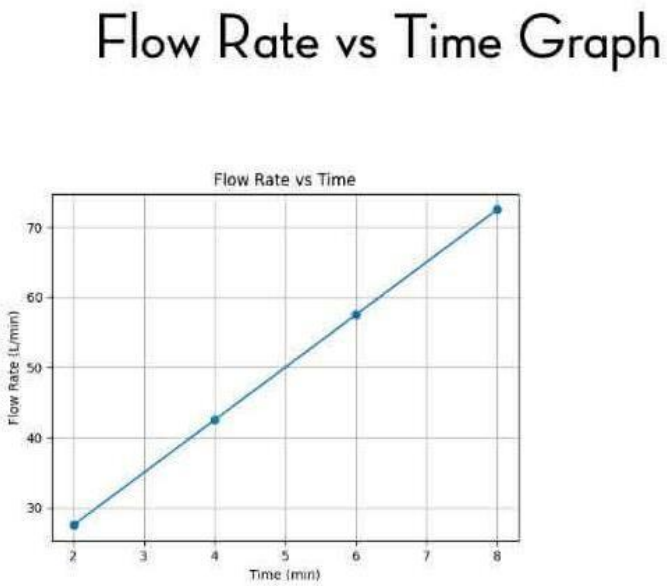


Source Code (Flow rate vs Time):

```
Import matplotlib.pyplot as plt

# FLOW RATE DATA
Time = [2, 4, 6, 8]
Flow_rate = [27.5, 42.5, 57.5, 72.5]

# CREATE GRAPH

Plt.figure()
Plt.plot(time, flow_rate, marker='o')

Plt.title("Flow Rate vs Time")
```

```
Plt.xlabel("Time (minutes)")
Plt.ylabel("Flow Rate (L/min)")
Plt.grid()

# SAVE IMAGE FILE
Plt.savefig("flow_rate_graph.png")

# DISPLAY GRAPH
Plt.show()
```

Output(image):



CASE STUDY 1: Population Growth Analysis Using Numerical Methods
Made by/Reporter: Tayba B. Abdullah

Scenario
A local government wants to analyze how population changes over time. However, they only have **yearly population data**, not a continuous function.

Students must:
• Estimate population growth rate (derivative)
• Estimate total population change (integration)
• Analyze trends

Computational Task

Given Data

Year	*Population*
2020	10,000
2021	10,800
2022	11,900
2023	13,200
2024	14,800

Steps

1. Numerical Differentiation (Growth Rate)
• Use central difference formula:

$$P'(t) \approx \frac{P(t+1) - P(t-1)}{2}$$

Students compute:
• Growth rate for 2021–2023

Compute:

- For 2021:

$$P'(2021) = \frac{11900 - 10000}{2} = 950$$

- For 2022:

$$P'(2022) = \frac{13200 - 10800}{2} = 1200$$

- For 2023:

$$P'(2023) = \frac{14800 - 11900}{2} = 1450$$

Growth Rate Table

Year	Growth Rate
2021	950
2022	1200
2023	1450

2. Numerical Integration (Total Population Change)

We use the **Trapezoidal Rule**:

$$\int_{2020}^{2024} P(t)\,dt$$

$$\int_a^b P(t)\,dt \approx \frac{h}{2}[P_0 + 2P_1 + 2P_2 + \ldots + P_n]$$

Where h = 1 year

$$= \frac{1}{2}[10000 + 2(10800 + 11900 + 13200) + 14800]$$

$$= \frac{1}{2}[10000 + 71800 + 14800]$$

$$= \frac{1}{2}(96600)$$

$$= 48,300$$

Step-by-step:

- Add middle values:

$$2(35900) = 71800$$

- Multiply by 2:

$$2(35900) = 71800$$

- Add first and last:

$$10000 + 14800 = 24800$$

- Add everything:

$$24800 + 71800 = 96600$$

- Divide by 2:

$$96600 \div 2 = 48,300$$

Total estimated population accumulation:

$$= 48,300$$

3. Visualization

Source code (python):

```
Welcome    population_analysis.py
C: > Users > Admin > population_analysis.py > ...
import numpy as np
import matplotlib.pyplot as plt

# Data
(variable) population: NDArray[Any]                    023, 2024])
population = np.array([10000, 10800, 11900, 13200, 14800])

# Central Difference (Growth Rate)
growth_years = years[1:-1]
growth_rates = (population[2:] - population[:-2]) / 2

# Trapezoidal Rule (MANUAL FIX - no np.trapz)
integral = 0
for i in range(len(years) - 1):
    integral += (population[i] + population[i+1]) / 2 * (years[i+1] - years[i])

# Output
print("Growth Rates:")
for y, g in zip(growth_years, growth_rates):
    print(f"Year {y}: {g}")

print("\nTotal Population Estimate (Integration):", integral)

# Plot Population
plt.figure()
plt.plot(years, population, marker='o')
plt.title("Population vs Time")
plt.xlabel("Year")
plt.ylabel("Population")
plt.grid()
```

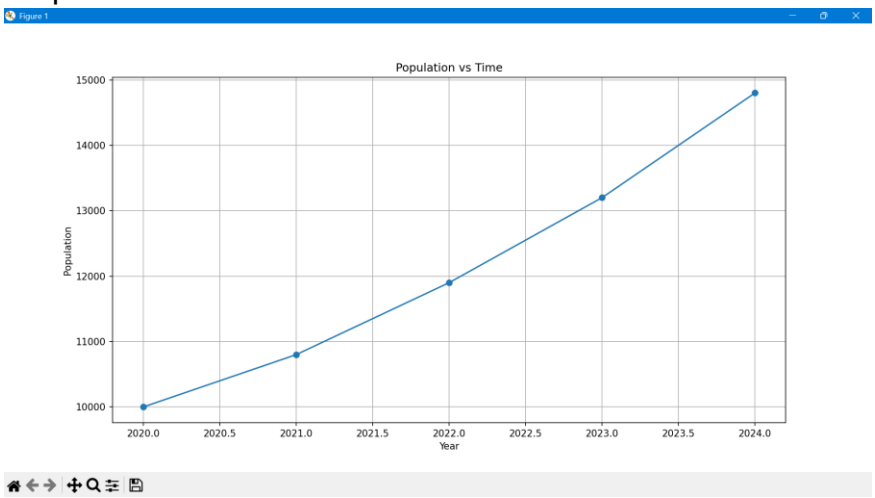
```
# Plot Growth Rate
plt.figure()
plt.plot(growth_years, growth_rates, marker='o')
plt.title("Growth Rate vs Time")
plt.xlabel("Year")
plt.ylabel("Growth Rate")
plt.grid()

plt.show()
```

Output:

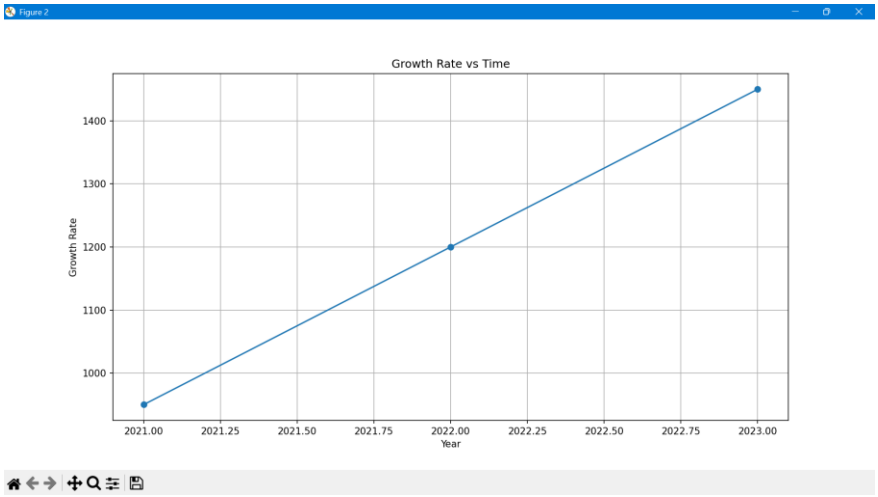
```
PS C:\Users\Admin\AppData\Local\Programs\Microsoft VS Code> & c:\python314\python.exe c:/Users/Admin/population_analysis.py
Growth Rates:
Year 2021: 950.0
Year 2022: 1200.0
Year 2023: 1450.0

Total Population Estimate (Integration): 48300.0
```

• Plot:

- ❑ Population vs time



- ❑ Growth rate vs time



4. Analysis Questions

- ➢ **When did growth accelerate the most?**

Answer:

The highest growth rate is in 2023 = 1450, so this is the year when the population increased the fastest.
This means, as time goes on, the population is growing at an increasing rate.

- ➢ **Is the growth linear or exponential?**

Answer:

The growth is not linear because:

- The increase is not constant
- The added population becomes larger each year

Therefore, the pattern suggests exponential growth, since the growth rate keeps increasing.

- **Prediction Population for 2025.**

Answer:

We can use the trend of increases:

- +800 (2020–2021)
- +1100
- +1300
- +1600

Average increase ≈ +1200 to +1500

Estimate:

$$2025 \approx 14800 + 1700 = 16,500$$

Predicted Population (2025): ~16,500

In conclusion, based on our computations, we observed that the population is not only increasing but accelerating over time. Using numerical differentiation, we found that the growth rate increases from 950 to 1450. Using numerical integration, the total population estimate from 2020 to 2024 is 48,300. Therefore, we conclude that the trend is exponential rather than linear. Based on this pattern, the projected population for 2025 is approximately 16,500.

CASE STUDY 2: Traffic Flow and Velocity Estimation
Made by/Reporter: Erika G. Uyasan

Scenario
A traffic monitoring system records the **position of a vehicle every second**. Engineers want to compute **velocity and total distance traveled**.

Computational Task

Students must:
- Estimate instantaneous velocity
- Compute total distance
- Analyze traffic behavior

Given Data

Time (s)	*Position (m)*
0	0
1	5
2	15
3	30
4	50
5	75

Steps

1. Velocity Estimation

- Use central difference:

$$v(t) = \frac{x(t+1) - x(t-1)}{2}$$

Compute velocity at:
- t = 1, 2, 3, 4

Time (s)	Position (m)	Velocity (m/s)
0	0	-
1	5	**7.5**
2	15	**12.5**
3	30	**17.5**
4	50	**22.5**
5	75	-

solving:

t = 1s

- t-1 = 0s (Position = 0)
- t+1 = 2s (Position = 15)

$$\mathrm{v}(1) = \frac{15-0}{2} = \frac{15}{2} = \mathbf{7.5}\boldsymbol{m/s}$$

t = 2s

- t-1 = 1s (Position = 5)
- t+1 = 3s (Position = 30)

$$\mathrm{v}(2) = \frac{30-5}{2} = \frac{25}{2} = \mathbf{12.5}\boldsymbol{m/s}$$

t = 3s

- t-1 = 2s (Position = 15)
- t+1 = 4s (Position = 50)

$$\mathrm{v}(3) = \frac{50-15}{2} = \frac{35}{2} = \mathbf{17.5}\boldsymbol{m/s}$$

t = 4s

- t-1 = 3s (Position = 30)
- t+1 = 5s (Position = 75)

$$\mathrm{v}(4) = \frac{75-30}{2} = \frac{45}{2} = \mathbf{22.5}\boldsymbol{m/s}$$

2. Acceleration Insight (Optional Extension)

• Apply differentiation again to velocity

Acceleration is the derivative of velocity. Based on the calculated velocities, the speed increases by exactly 5 m/s every second:

- 12.5 - 7.5 = 5
- 17.5 - 12.5 = 5
- 22.5 - 17.5 = 5

This indicates a constant acceleration of $\mathbf{5}\boldsymbol{m/s^2}$**.**

3. Distance Verification

• Use Trapezoidal Rule to compute:

$$\int v(t)\, dt$$

Distance ≈ $\frac{\Delta t}{2}[v(t1) + 2v(t2) + 2v(t3) + v(t4)]$

Distance ≈ $\frac{1}{2}[7.5 + 2(12.5) + 2(17.5) + 22.5]$

Distance ≈ $\frac{1}{2}[7.5 + 25 + 35 + 22.5] = \frac{90}{2} = \mathbf{45m}$

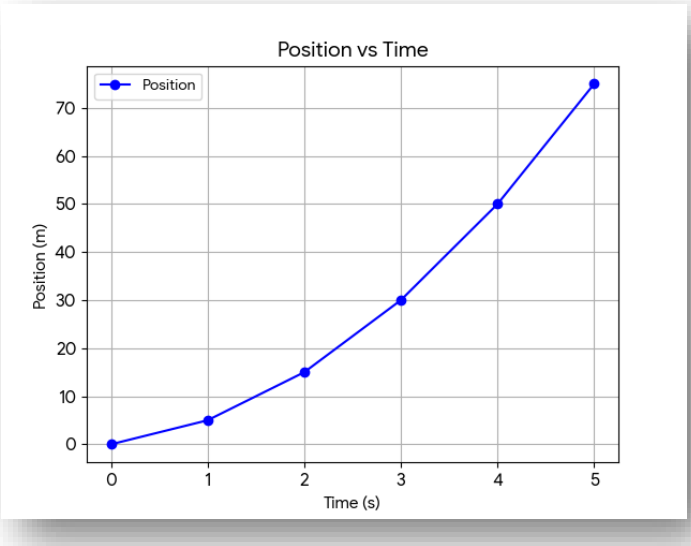
Compare with actual displacement.

The actual displacement between t = 1 and t = 4 is x(4) - x(1) = 50 - 5 = **45 m**. The numerical estimation is perfectly accurate because the velocity increases linearly.

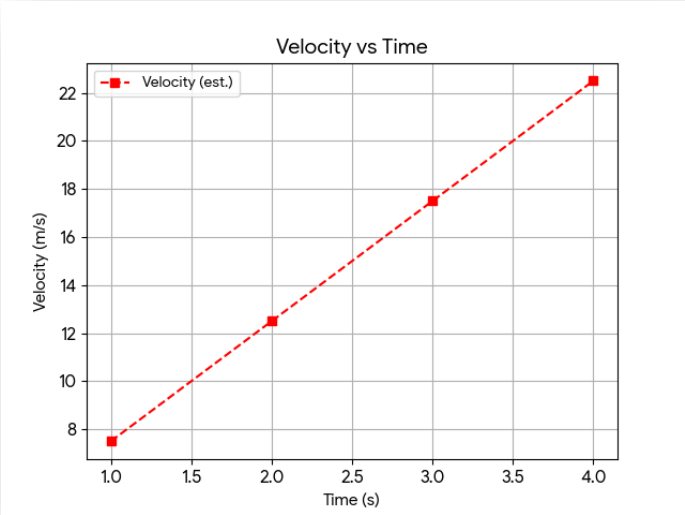
4. Visualization

• Plot:

- Position vs time



- Velocity vs time



Python Source Code:

```
import matplotlib.pyplot as plt
import numpy as np

# --- 1. SET UP THE DATA ---
time = np.array([0, 1, 2, 3, 4, 5])
position = np.array([0, 5, 15, 30, 50, 75])

# Calculate velocity using Central Difference Method
# v(t) = [x(t+1) - x(t-1)] / 2
v_time = time[1:5]
velocity = (position[2:6] - position[0:4]) / 2

# --- 2. GENERATE POSITION VS TIME (Blue Curve) ---
plt.figure(figsize=(8, 6))
plt.plot(time, position, color='blue', marker='o', linestyle='-', label='Position')

plt.title('Position vs Time', fontsize=14)
plt.xlabel('Time (s)', fontsize=12)
plt.ylabel('Position (m)', fontsize=12)
plt.grid(True)
plt.legend(loc='upper left')

# To show it on screen or save it
plt.savefig('Position_vs_Time.png')
plt.show()
```

```
# --- 3. GENERATE VELOCITY VS TIME (Red Dashed Line) ---
plt.figure(figsize=(8, 6))
plt.plot(v_time, velocity, color='red', marker='s', linestyle='--', label='Velocity (est.)')

plt.title('Velocity vs Time', fontsize=14)
plt.xlabel('Time (s)', fontsize=12)
plt.ylabel('Velocity (m/s)', fontsize=12)
plt.grid(True)
plt.legend(loc='upper left')

# To show it on screen or save it
plt.savefig('Velocity_vs_Time.png')
plt.show()
```

5. Analysis Questions

- **When is the car accelerating?**

The car is accelerating throughout the entire duration (0 to 5 seconds). As we observation, the distance covered in each one-second interval is increasing (5, 10, 15, 20, 25 meters respectively).

- **Is motion uniform?**

No**.** Uniform motion requires a constant velocity. Since the velocity is increasing, this is a Uniformly Accelerated Motion where the velocity changes by the same amount in every equal time interval.

- **Identify any anomalies (sudden jumps)**

There are no anomalies or sudden jumps in the data. The position values follow a smooth quadratic pattern, $x(t) = 2.5t^2 + 2.5t$, suggesting a very steady increase in speed.